

ARTIFICIAL INTELLIGENCE LAB 01

State Space Representation, BFS and DFS

Case Study: CampusCourier at a Sports Gala

Background

The university is hosting its annual Mini Olympics, and due to multiple sports venues, food stalls and heavy crowds, and temporary barricades, some corridors in the campus are blocked.

An AI-powered delivery robot, **CampusCourier**, has to deliver sports equipment from the **Storage Room** to the **Campus Ground** as quickly as possible. CampusCourier can move in four directions—up, down, left, or right—and cannot pass through blocked areas.

Your task is to model this problem as a state space and implement two uninformed search algorithms:

- Breadth-First Search (BFS)
 - Depth-First Search (DFS)
-

Problem Representation

Input File:

The campus grid is provided in a text file called campus.txt. The layout of the file is as follows:

- The first line of the file contains two numbers: **the number of rows** and **the number of columns** in the hospital layout.
 - The other lines represent the campus grid itself, with each cell separated by a space.
 - The symbols in the grid indicate the following:
 - S marks the Storage Room
 - G marks the Campus Ground
 - . represents an open space where CampusCourier can move
 - # represents a blocked corridor that CampusCourier cannot enter
-

Tasks

Task 1: Define the State Space

Formally define methods to return:

- State (current position of CampusCourier as coordinates)
- Initial State
- Goal Test
- Actions (all possible moves CampusCourier can make from the current state)
- Transition Function (takes a state and an action, and returns the **resulting state** after performing that action)

Note:

- You can represent the actions as a list of coordinate changes, like $[(-1, 0), (1, 0), (0, -1), (0, 1)]$ for Up, Down, Left, Right respectively.
 - If an action is invalid (e.g., moves into a wall or outside the grid), the transition function should not return that state.
-

Task 2: Implement BFS

Your BFS implementation must:

- Find a path from S to G
 - Return the full path
 - Count number of nodes explored
 - Return path length
-

Task 3: Implement DFS

Your DFS implementation must:

- Use a stack
 - Return a path from S to G
 - Count number of nodes explored
 - Return path length
-

Task 4: Output File

Write results to a file named result.txt.

Example output format for BFS and DFS:

Algorithm: BFS

Path: (0,0) -> (0,1) -> (0,2) -> ...

Path Length: 7

Nodes Explored: 12

Algorithm: DFS

Path: ...

Path Length: 10

Nodes Explored: 9

(Please choose a suitable output format for Task 1)

Submission Requirements

Submit all of the following in a zipped folder named **BSEF24M500_AI_Lab_01**:

1. Python file (.py)
2. Jupyter Notebook (.ipynb)
3. Input file (campus.txt)
4. Output file (result.txt)

Note: Plagiarism, use of generative AI, or copying from others shall be strictly penalized.
